

JAVA COURSE

{ condizionali e cicli }



INDICE

01 Le condizioni

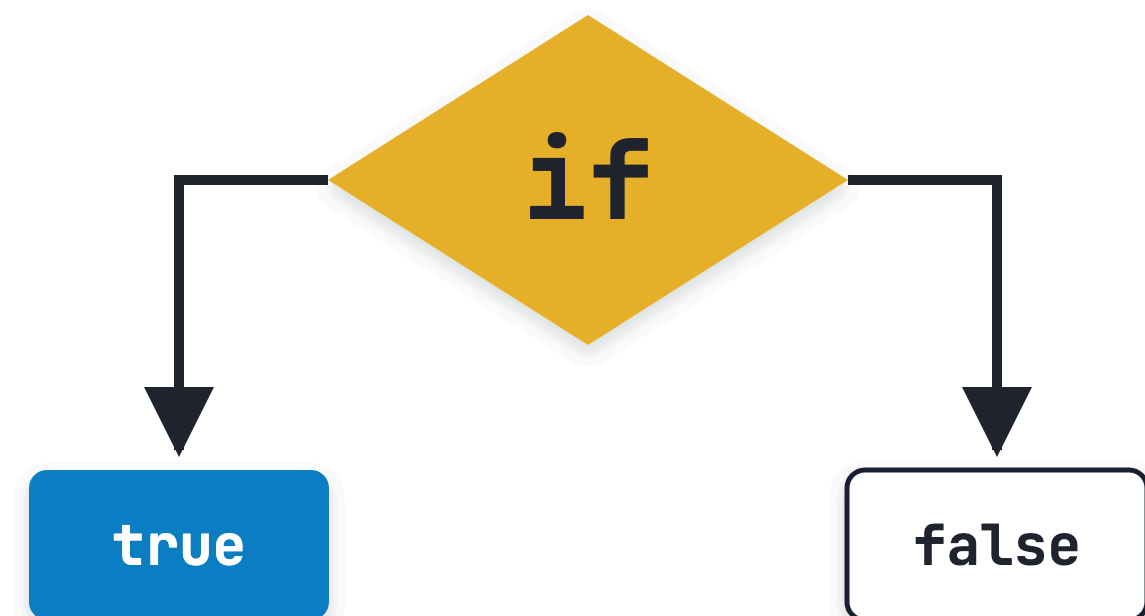
02 Lo switch

03 I cicli

04 Controllare i cicli

01

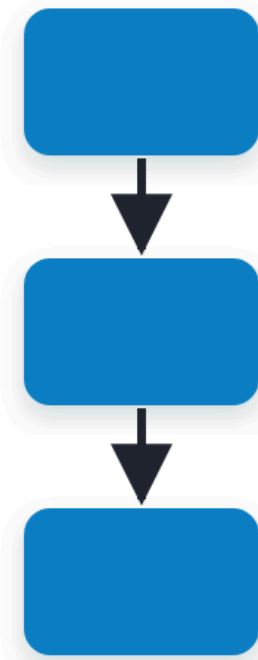
LE CONDIZIONI



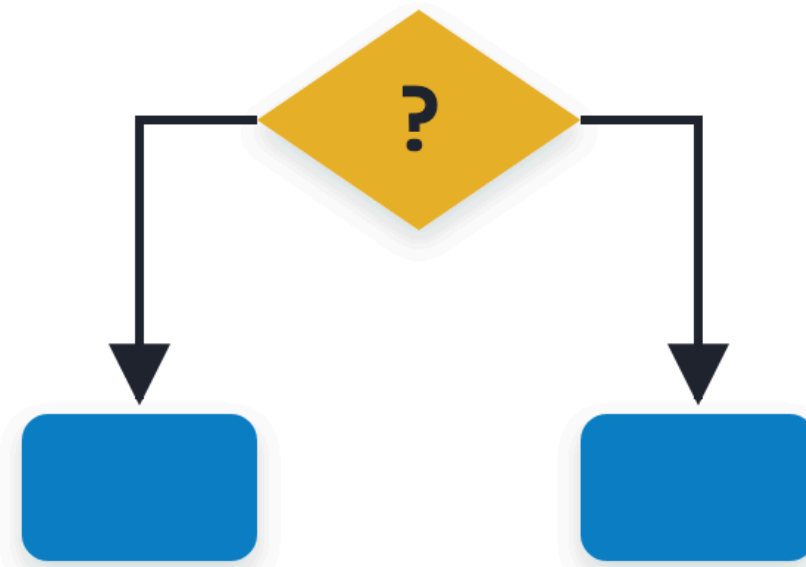
■ IL FLUSSO DEL PROGRAMMA

Finora le istruzioni venivano eseguite una dopo l'altra, dall'alto in basso. Le espressioni booleane permettono di cambiare questo ordine: **scegliere** quale codice eseguire e **ripetere** un blocco

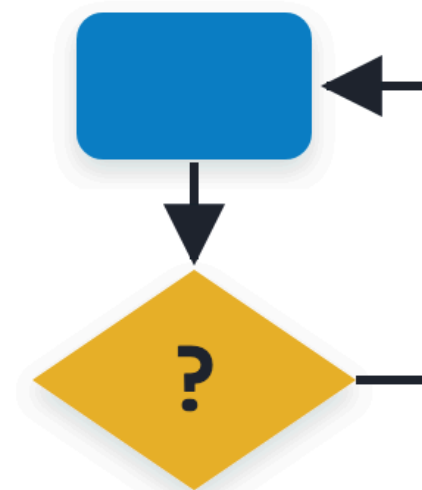
Sequenza



Scelta



Ripetizione



■ I BLOCCHI

Un **blocco** è un gruppo di istruzioni tra parentesi graffe. Una variabile dichiarata dentro un blocco esiste solo lì: è il suo **scope**

```
int eta = 20;
{
    int anno = 2026 - eta;
    System.out.println(anno);    // 2006
}
System.out.println(anno);      // errore: cannot find symbol
```

■ L' ISTRUZIONE IF

`if` esegue il blocco solo se la condizione è `true`. La condizione si scrive tra parentesi tonde ed è un'espressione di tipo `boolean`

```
int temperatura = 32;  
if (temperatura > 30) {  
    System.out.println("Fa caldo");  
}  
System.out.println("Fine");
```

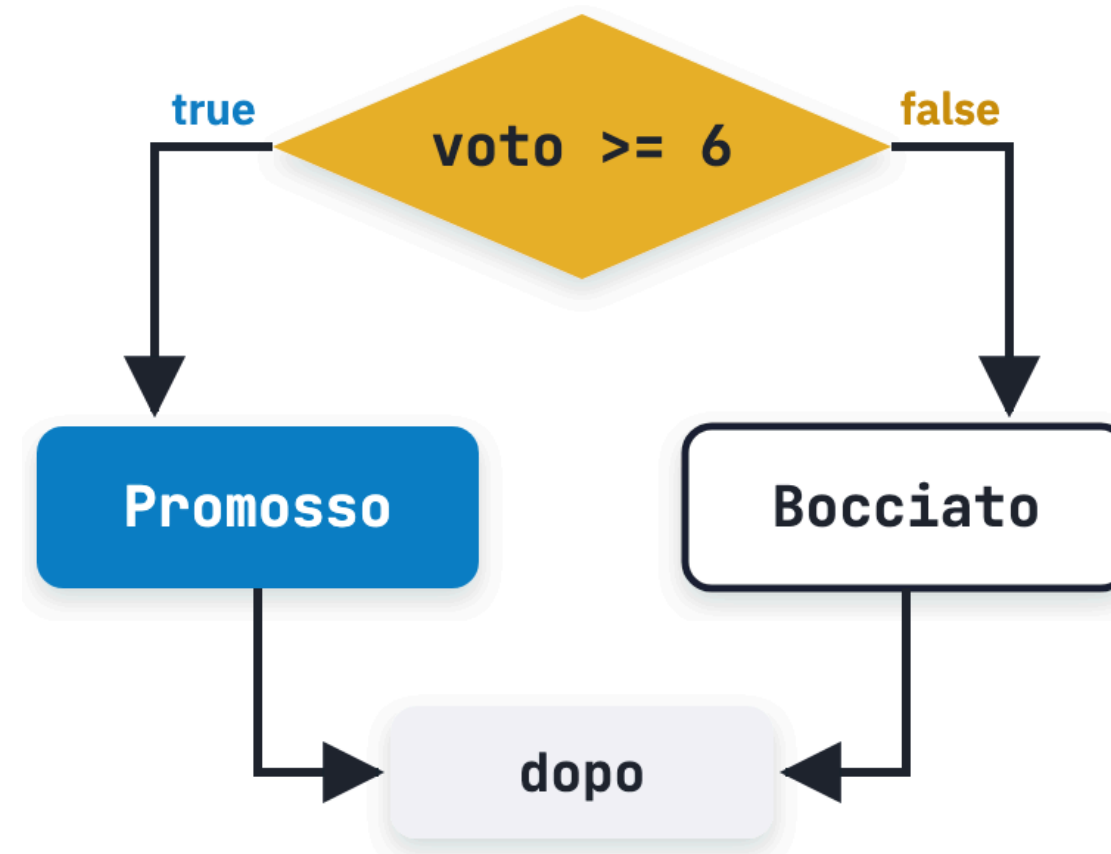
● ● ● Terminale

```
Fa caldo  
Fine
```

■ IF ED ELSE

else indica il blocco da eseguire quando la condizione è **false**: dei due blocchi ne viene eseguito sempre uno, e uno solo

```
int voto = 5;
if (voto >= 6) {
    System.out.println("Promosso");
} else {
    System.out.println("Bocciato");
}
// Bocciato
```



■ ELSE IF

Per più di due casi si concatenano `if` ed `else`: le condizioni si controllano dall'alto e si esegue solo il primo blocco vero

- *L'ordine conta: si parte dalla condizione più restrittiva*

```
int voto = 8;
if (voto >= 9) {
    System.out.println("Ottimo");
} else if (voto >= 7) {
    System.out.println("Buono");
} else if (voto >= 6) {
    System.out.println("Sufficiente");
} else {
    System.out.println("Insufficiente");
}
// Buono
```

■ CONDIZIONI COMPOSITE

Le condizioni si combinano con `&&` e `||`. Un intervallo si scrive con due confronti uniti da `&&`

```
int eta = 20;
boolean studente = true;
if (eta >= 18 && eta <= 65) {
    System.out.println("Biglietto intero");
}
if (eta < 26 || studente) {
    System.out.println("Sconto giovani");
}
// Biglietto intero
// Sconto giovani
```

■ IF ANNIDATI

Un `if` può contenere altri `if`. Oltre due livelli il codice si legge male: spesso una condizione composta è più chiara

```
int eta = 20;
boolean patente = true;
if (eta >= 18) {
    if (patente) {
        System.out.println("Può guidare");
    }
}
if (eta >= 18 && patente) {    // stesso risultato
    System.out.println("Può guidare");
}
```

■ IF: CHE COSA STAMPA?

Che cosa stampa questo codice?

```
int x = 15;
if (x > 10) {
    System.out.println("A");
} else if (x > 5) {
    System.out.println("B");
}
if (x % 2 == 0) {
    System.out.println("C");
} else {
    System.out.println("D");
}
// ??
```

■ L'OPERATORE TERNARIO

`condizione ? seVero : seFalso` è un'espressione: produce un valore, che si può assegnare o stampare. È la forma breve di un `if-else` che sceglie tra due valori

- *Ternari annidati diventano presto illeggibili: in quel caso meglio un `if`*

```
int eta = 20;
String fascia = eta >= 18 ? "maggiorenne" : "minorenne";
System.out.println(fascia);           // maggiorenne

int mele = 1;
System.out.println(mele + (mele == 1 ? " mela" : " mele")); // 1 mela
```

■ TERNARIO 0 IF

Il ternario sceglie un valore, l'`if` sceglie quali istruzioni eseguire. Per questo il ternario non può contenere istruzioni come `System.out.println`

```
boolean ok = true;  
ok ? System.out.println("sì") : System.out.println("no");    // errore: not a statement
```

■ ERRORI CLASSICI: IL PUNTO E VIRGOLA

Un punto e virgola subito dopo la condizione chiude l'`if` con un'istruzione vuota. Il blocco che segue non dipende più dalla condizione e viene eseguito sempre

```
int x = 3;
if (x > 10);
{
    System.out.println("Maggiore di 10"); // stampato comunque
}
```

■ ERRORI CLASSICI: LE GRAFFE

Senza graffe l'`if` controlla solo l'istruzione successiva. L'indentazione inganna: per il compilatore non conta

- *Le graffe si scrivono sempre, anche per una sola istruzione*

```
int x = 3;  
if (x > 10)  
    System.out.println("Maggiore di 10");  
    System.out.println("Anche questa?"); // stampata sempre
```

■ ERRORI CLASSICI: I CONFRONTI

`=` assegna, `==` confronta: un'assegnazione nella condizione è un errore, perché la condizione deve essere un `boolean`

```
int x = 3;
if (x = 5) {           // errore: incompatible types: int cannot be converted to boolean
    System.out.println("cinque");
}
```

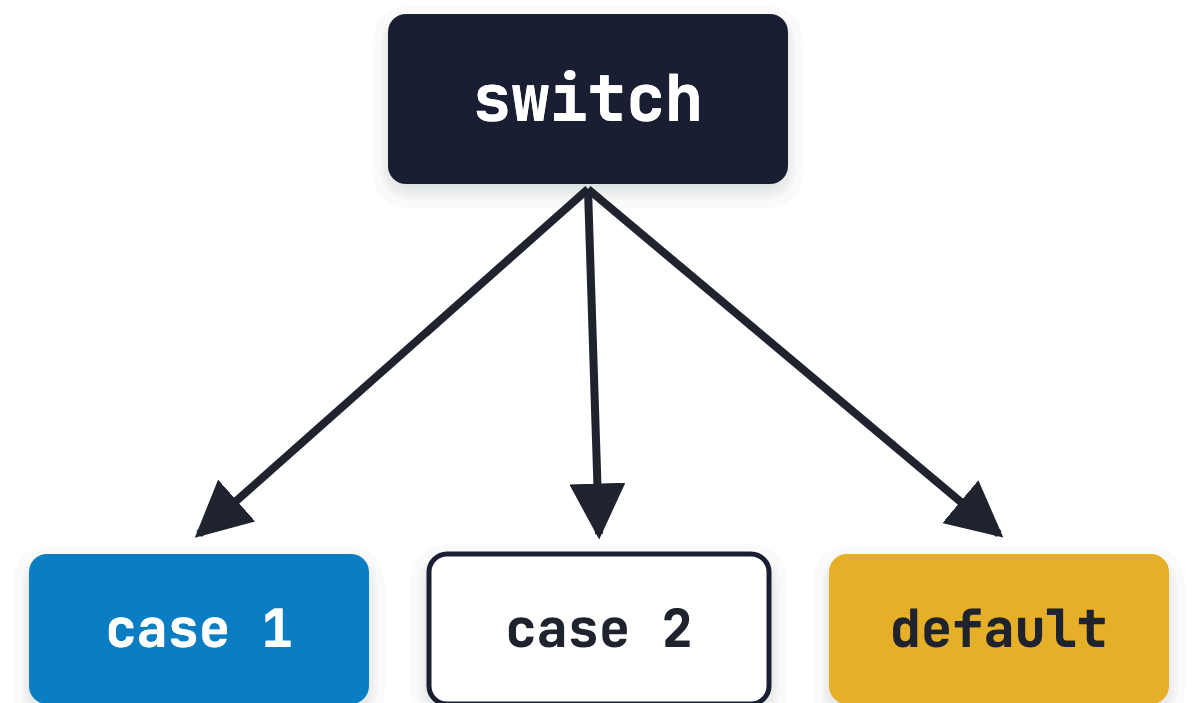
■ ERRORI CLASSICI: LE STRINGHE

Il test di due stringhe si confronta con `equals`. `==` confronta i riferimenti, e può dare `false` anche con lo stesso testo

```
String parte = "Ja";
String linguaggio = parte + "va";
if (linguaggio == "Java") {           // false: confronta i riferimenti
    System.out.println("Uguale con ==");
}
if (linguaggio.equals("Java")) {      // true: confronta il testo
    System.out.println("Uguale con equals");
}
```

02

L0 SWITCH



■ SWITCH

`switch` confronta un valore con una serie di casi e salta al `case` corrispondente. `default` si esegue quando nessun caso corrisponde

- Funziona con `byte`, `short`, `int`, `char`, i rispettivi wrapper, `enum` e `String`; non con `double` e `boolean`

```
int giorno = 3;
switch (giorno) {
    case 1:
        System.out.println("Lunedì");
        break;
    case 3:
        System.out.println("Mercoledì");
        break;
    default:
        System.out.println("Giorno non valido");
}
// Mercoledì
```

■ IL FALL-THROUGH

Senza `break` l'esecuzione prosegue nei `case` successivi, anche se non corrispondono: è il **fall-through**, più spesso un errore che una scelta

```
int livello = 2;
switch (livello) {
    case 1:
        System.out.println("Base");
    case 2:
        System.out.println("Intermedio");
    case 3:
        System.out.println("Avanzato");
        break;
}
// Intermedio
// Avanzato
```

■ CASI RAGGRUPPATI

Il fall-through serve a dare lo stesso risultato a più valori

```
int mese = 2;
int giorni;
switch (mese) {
    case 4: case 6: case 9: case 11:
        giorni = 30;
        break;
    case 2:
        giorni = 28;
        break;
    default:
        giorni = 31;
}
System.out.println(giorni);    // 28
```

■ SWITCH: CHE COSA STAMPA?

Che cosa stampa questo codice?

```
char c = 'b';
switch (c) {
    case 'a':
        System.out.print("A");
    case 'b':
        System.out.print("B");
    case 'c':
        System.out.print("C");
        break;
    default:
        System.out.print("D");
}
// ??
```

■ LA FORMA CON LA FRECCIA

Da Java 14 i casi si scrivono anche con la freccia `->`: ogni caso esegue solo il suo codice, senza `break` né fall-through

- *Le due forme non si mescolano nello stesso `switch`: javac segnala `different case kinds used in the switch`*

```
int giorno = 6;
switch (giorno) {
    case 1, 2, 3, 4, 5 -> System.out.println("Feriale");
    case 6, 7 -> System.out.println("Weekend");
    default -> System.out.println("Non valido");
}
// Weekend
```

■ SWITCH COME ESPRESSIONE

Con la freccia lo `switch` può produrre un valore. Come espressione deve essere **esaustivo**: deve coprire tutti i valori possibili

- ***default**: obbligatorio con `int` e `String`; con un `enum` si può omettere se si coprono tutti i valori*

```
int giorno = 6;
String tipo = switch (giorno) {
    case 1, 2, 3, 4, 5 -> "feriale";
    case 6, 7 -> "weekend";
    default -> "non valido";
};
System.out.println(tipo);    // weekend
```

■ YIELD

Quando un caso ha bisogno di più istruzioni, si scrive un blocco tra graffe e il valore si restituisce con `yield`

```
int punti = 75;
String giudizio = switch (punti / 10) {
    case 10, 9 -> "ottimo";
    case 8, 7 -> {
        System.out.println("Buon risultato");
        yield "buono";
    }
    default -> "da migliorare";
};
System.out.println(giudizio);
// Buon risultato
// buono
```

■ SWITCH SU STRING

Lo `switch` funziona anche con le stringhe: il confronto avviene sul testo, come con `equals`

- *Maiuscole e minuscole contano: `"Stop"` non corrisponde a `"stop"`*

```
String comando = "stop";
switch (comando) {
    case "start" -> System.out.println("Avvio");
    case "stop" -> System.out.println("Arresto");
    default -> System.out.println("Comando sconosciuto");
}
// Arresto
```

■ LE DUE FORME A CONFRONTO

La forma con la freccia è più breve ed evita il fall-through per sbaglio. La forma classica resta valida ed è diffusa nel codice esistente

	Forma classica	Con la freccia
Fall-through	sì, senza <code>break</code>	no
Più valori	<code>case 4: case 6:</code>	<code>case 4, 6 -></code>
Produce un valore	no	sì, come espressione
Disponibile	da sempre	da Java 14

■ IL PATTERN MATCHING

Da Java 21 un `case` può controllare anche il **tipo** di un valore e aggiungere una condizione con `when`. È il pattern matching, utile soprattutto con gli oggetti

```
Object valore = 42;
String descrizione = switch (valore) {
    case Integer n when n > 10 -> "numero grande";
    case Integer n -> "numero piccolo";
    case String t -> "testo";
    default -> "altro";
};
System.out.println(descrizione);    // numero grande
```

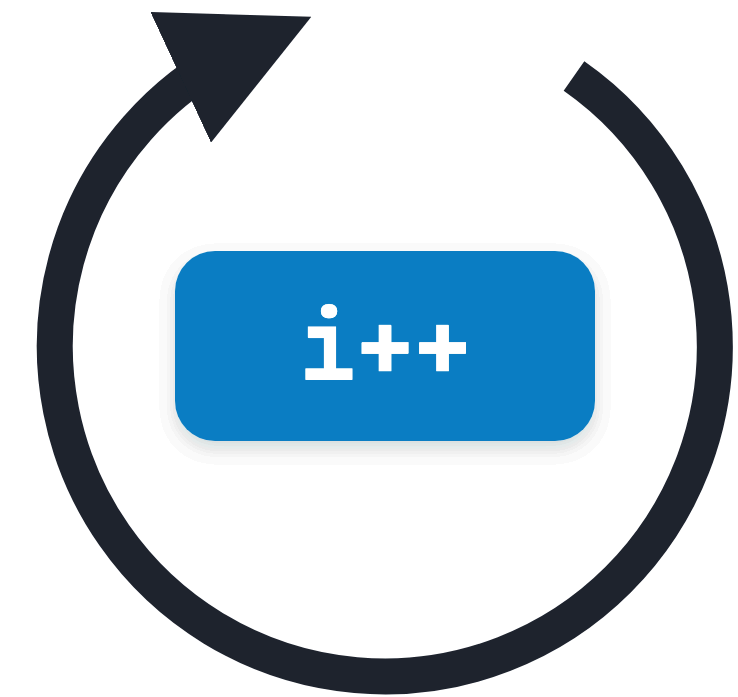
■ JAVA 21 E JAVA 25

Per lo `switch` le due versioni si comportano allo stesso modo. La novità di Java 25, i pattern sui tipi primitivi, è ancora in anteprima e va abilitata con `--enable-preview`

Funzionalità	Java 21	Java 25
Freccia e switch come espressione	sì	sì
Pattern matching con <code>when</code>	sì	sì
Pattern sui primitivi, <code>case int i</code>	no	in anteprima

03

I CICLI



■ RIPETERE

Un **ciclo** ripete un blocco di istruzioni; ogni ripetizione si chiama **iterazione**.

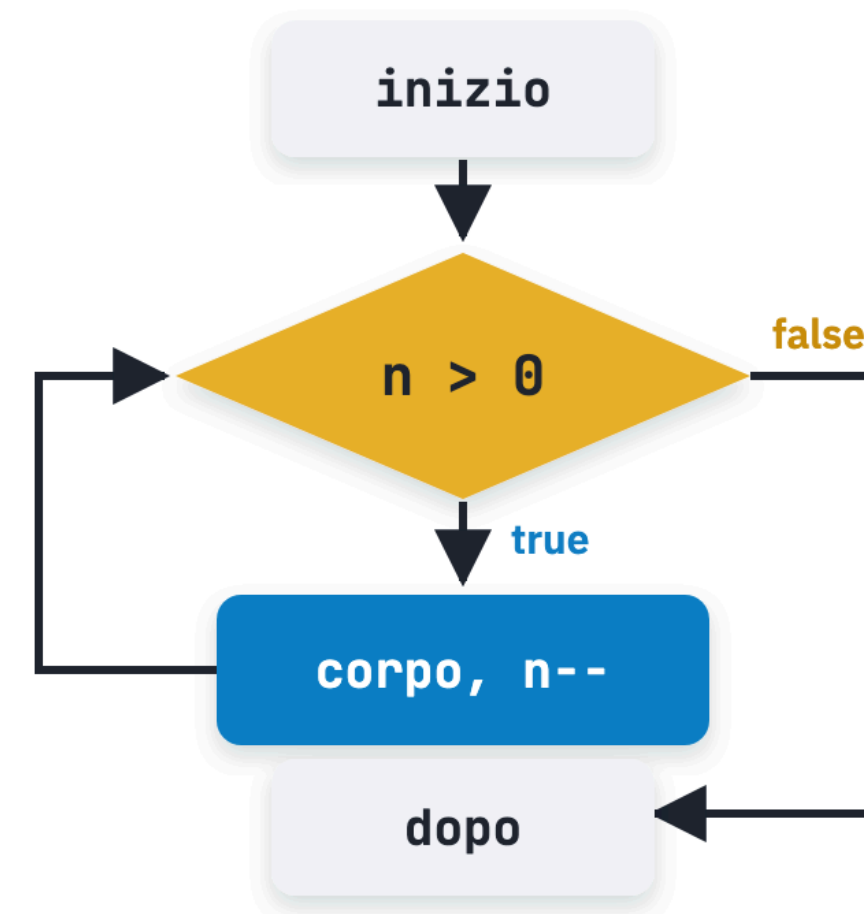
Ripetere senza errori è ciò che i computer fanno meglio

Ciclo	Quando si usa
<code>while</code>	finché una condizione è vera, senza sapere quanti giri servono
<code>do-while</code>	come <code>while</code> , ma il primo giro si fa sempre
<code>for</code>	quando il numero di giri è noto, con un contatore

■ WHILE

while ripete il blocco finché la condizione è **true**, e la controlla prima di ogni giro: se è falsa subito, il blocco non viene mai eseguito

```
int n = 3;  
while (n > 0) {  
    System.out.println(n);  
    n--;  
}  
System.out.println("Via!");
```



■ IL CICLO INFINITO

Il corpo del ciclo deve portare la condizione verso `false`. Se non succede mai, il ciclo non finisce: è un **ciclo infinito**, e il programma resta bloccato

■ *Dal terminale si interrompe con Ctrl + C*

```
int n = 3;
while (n > 0) {
    System.out.println(n);    // n non cambia mai: stampa 3 per sempre
}
```



■ DO-WHILE

`do-while` controlla la condizione dopo il giro: il blocco viene eseguito almeno una volta, anche se la condizione è falsa dall'inizio

■ *Dopo la condizione serve il punto e virgola:* `} while (n < 5);`

```
int n = 10;
do {
    System.out.println("Giro con n = " + n);
    n++;
} while (n < 5);
// Giro con n = 10
```

■ FOR

for raccoglie in una riga le tre parti di un ciclo con contatore: l'inizio, la condizione e l'aggiornamento

for (int i = 1; i <= 5; i++)

 inizio condizione aggiornamento

```
for (int i = 1; i <= 5; i++) {  
    System.out.print(i + " ");  
}  
// 1 2 3 4 5
```

■ COME SI ESEGUE UN FOR

L'inizio si esegue una volta sola. Poi, a ogni giro: si controlla la condizione, si esegue il corpo, si esegue l'aggiornamento

Giro	i	i <= 3	Stampa
1	1	true	1
2	2	true	2
3	3	true	3
fine	4	false	—

■ LO SCOPE DEL CONTATORE

Una variabile dichiarata nell'intestazione del `for` esiste solo dentro il ciclo.
Se serve anche dopo, si dichiara prima del ciclo

```
for (int i = 0; i < 3; i++) {  
    System.out.println(i);  
}  
System.out.println(i);    // errore: cannot find symbol
```

■ PASSI DIVERSI

L'aggiornamento non deve essere per forza `i++`: si può contare all'indietro o a passi diversi

```
for (int i = 10; i > 0; i -= 3) {  
    System.out.print(i + " ");    // 10 7 4 1  
}  
System.out.println();  
for (int i = 1; i <= 1000; i *= 10) {  
    System.out.print(i + " ");    // 1 10 100 1000  
}
```

■ FOR: CHE COSA STAMPA?

Che cosa stampa questo ciclo?

```
for (int i = 0; i < 10; i += 4) {  
    System.out.print(i + " ");  
}  
// ??
```

■ CONTATORI E ACCUMULATORI

Un **contatore** aumenta di 1 quando succede qualcosa, un **accumulatore** raccoglie un totale. Entrambi si inizializzano prima del ciclo

```
int somma = 0;
int pari = 0;
for (int i = 1; i <= 10; i++) {
    somma += i;
    if (i % 2 == 0) {
        pari++;
    }
}
System.out.println(somma);    // 55
System.out.println(pari);    // 5
```

■ IL MASSIMO

Si tiene il valore più grande visto finora e lo si aggiorna quando ne arriva uno maggiore. Le cifre si leggono con `%` e `/`

```
int numero = 58291;
int massimo = 0;
while (numero > 0) {
    int cifra = numero % 10;
    if (cifra > massimo) {
        massimo = cifra;
    }
    numero /= 10;
}
System.out.println(massimo);    // 9
```

■ CICLI ANNIDATI

Un ciclo può contenerne un altro: per ogni giro del ciclo esterno, il ciclo interno fa tutti i suoi giri

```
for (int riga = 1; riga <= 3; riga++) {  
    for (int col = 1; col <= 4; col++) {  
        System.out.print(riga * col + " ");  
    }  
    System.out.println();  
}
```

● ● ● Terminale

```
1 2 3 4  
2 4 6 8  
3 6 9 12
```

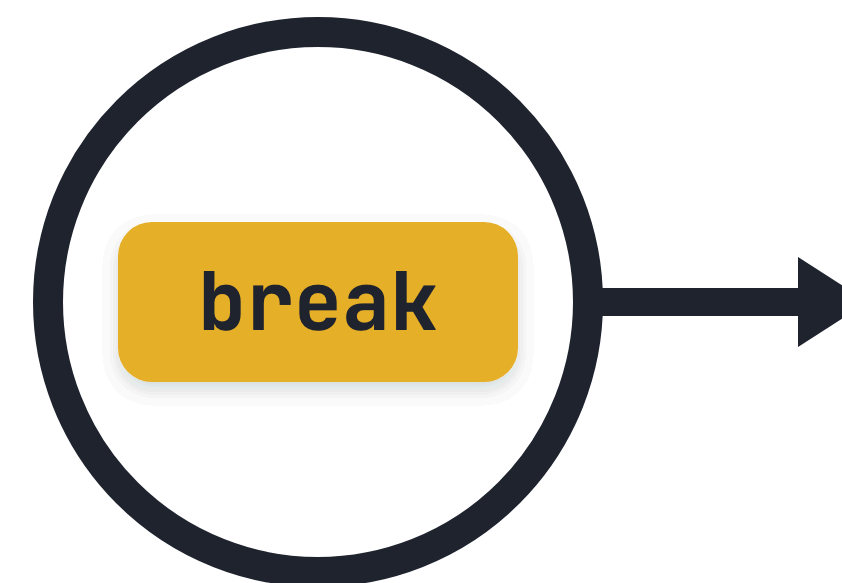
■ ANNIDATI: CHE COSA STAMPA?

Che cosa stampano i due cicli?

```
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= i; j++) {  
        System.out.print("*");  
    }  
    System.out.println();  
}  
// ??
```

04

CONTROLLARE I CICLI



■ BREAK

break interrompe subito il ciclo e passa all'istruzione che lo segue. Serve per uscire appena si è trovato ciò che si cercava

```
int numero = 91;
for (int d = 2; d < numero; d++) {
    if (numero % d == 0) {
        System.out.println("Primo divisore: " + d);
        break;
    }
}
// Primo divisore: 7
```

■ CONTINUE

`continue` salta il resto del giro corrente e passa al successivo. In un `for` l'aggiornamento viene eseguito comunque

```
for (int i = 1; i <= 10; i++) {  
    if (i % 3 == 0) {  
        continue;  
    }  
    System.out.print(i + " ");  
}  
// 1 2 4 5 7 8 10
```

■ BREAK NEI CICLI ANNIDATI

break esce solo dal ciclo più interno in cui si trova: il ciclo esterno continua

```
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 3; j++) {  
        if (j == 2) {  
            break;  
        }  
        System.out.print(i + "-" + j + " ");  
    }  
}
```

// 1-1 2-1 3-1

■ BREAK E CONTINUE: CHE COSA STAMPA?

Che cosa stampa questo ciclo?

```
int i = 0;
while (i < 10) {
    i++;
    if (i % 2 == 0) {
        continue;
    }
    if (i > 7) {
        break;
    }
    System.out.print(i + " ");
}
// ??
```

■ CICLI INFINITI VOLUTI

`while (true)` e `for (;;)` non finiscono da soli. Si usano quando la condizione di uscita sta a metà del ciclo, e si chiudono con `break`

```
int n = 0;
while (true) {
    n++;
    if (n * n > 50) {
        break;
    }
}
System.out.println(n); // 8
```



■ QUALE CICLO SCEGLIERE

Ogni ciclo si può riscrivere con gli altri, ma ognuno è il più chiaro in una situazione

Situazione	Ciclo
Il numero di giri è noto	<code>for</code>
Si ripete finché succede qualcosa	<code>while</code>
Il primo giro si fa sempre	<code>do-while</code>
L'uscita sta a metà del ciclo	<code>while (true)</code> con <code>break</code>

■ CONFRONTO CON PYTHON

Differenze su condizionali e cicli rispetto a Python

Aspetto	Python	Java
Blocchi	indentazione	parentesi graffe
Condizione	<code>if x > 10:</code>	<code>if (x > 10) {</code>
elif	<code>elif</code>	<code>else if</code>
Ternario	<code>"sì" if ok else "no"</code>	<code>ok ? "sì" : "no"</code>
switch	<code>match/case</code> (3.10+)	<code>switch</code> classico e con freccia
for	<code>for i in range(5)</code>	<code>for (int i = 0; i < 5; i++)</code>
while/else	esiste	non esiste

■ ERRORI COMUNI

I messaggi reali di `javac`

Errore	Messaggio di javac
<code>;</code> dopo la condizione dell' <code>if</code>	compila, ma il blocco si esegue sempre
<code>=</code> al posto di <code>==</code>	<code>incompatible types: int cannot be converted to boolean</code>
Stringhe con <code>==</code>	compila, ma confronta i riferimenti
Switch non esaustivo	<code>the switch expression does not cover all possible input values</code>
Variabile del <code>for</code> fuori dal ciclo	<code>cannot find symbol</code>
Variabile forse non inizializzata	<code>variable might not have been initialized</code>

DOMANDE?

